Sistema de Reservas de Salas

Nombre: Jesús Artiaga Fau

Fecha: 02/03/2026

Información Técnica

Versión: 1.0

Version Git: git version 2.52.0.windows.1

Tecnologías utilizadas:

- Java 17
- Spring Boot (framework principal)
- Spring Web (exposición de endpoints REST)
- Spring Data JPA (acceso a datos)
- Hibernate (ORM)
- H2 Database (base de datos en memoria para desarrollo y pruebas)
- URL JDBC utilizada durante la ejecución: jdbc:h2:mem:7d5b175a-3878-4242-bece-ec5d7f7eb175 *por ejemplo va cambiando en función apagues y enciendas el api*
- Bean Validation (Jakarta Validation): @NotBlank, @Email, @Positive, @Future, etc.

1. Introducción

El proyecto **Gestión de reservas de salas** se encarga de administrar las salas de reuniones dentro de una organización (Empresas, Centros educativos, Entornos sanitarios etc). En muchos entornos empresariales, la asignación de salas se realiza manualmente o mediante herramientas poco estructuradas, lo que genera solapamientos, falta de control y dificultades para coordinar la disponibilidad entre equipos. Esta API proporciona un sistema robusto que evita estos problemas mediante reglas de negocio claras y un flujo de reservas controlado.

La aplicación ofrece funcionalidades esenciales como:

- Creación, eliminación y consulta de **salas**.
- Creación, eliminación y consulta de **usuarios**.
- Creación, eliminación y consulta de una **reserva**, incluyendo su confirmación y cancelación.

Además, garantiza la integridad del sistema mediante validaciones como:

- Usuario, Sala o Usuario enviado donde no corresponde
- Datos desconocidos
- JSON mal formado
- Tipo correcto pero estructura incorrecta (campos mal escritos)
- La fecha de inicio no puede ser igual a la fecha de fin
- No se puede editar o confirmar una reserva que ya ha comenzado
- No se puede cancelar una reserva que ya ha finalizado
- No se puede confirmar una reserva ya confirmada
- No se puede cancelar una reserva ya cancelada
- No se puede editar o borrar una reserva confirmada
- La fecha de inicio debe ser anterior a la fecha de fin
- No puede haber solapamiento de reservas en la misma sala
- Reserva no encontrada (ID inexistente)
- Fechas inválidas y nulas
- Solapamiento de reservas
- Reserva no encontrada
- Error interno del servidor

Se trata de una API REST desarrollada con Java y Spring Boot, estructurada en capas:

- Controladores, que exponen los endpoints REST y gestionan las peticiones del cliente.
- Servicios, donde reside la lógica de negocio, incluyendo validaciones, reglas de dominio y control de estados.
- Repositorios, encargados del acceso a la base de datos mediante JPA.
- Modelo de dominio, que define las entidades y sus relaciones (Reserva, Sala, Usuario, StatusReserva).
- DTOs, utilizados para intercambiar datos sin exponer directamente las entidades internas.
- Mappers, responsables de convertir entidades ↔ DTOs de forma consistente.
- Excepciones personalizadas y manejador global, que aseguran respuestas coherentes (400, 404, 409, etc.).
- Configuración, donde se definen interceptores y reglas adicionales para validar el tipo de datos recibido.

Todo para asegurar modularidad, mantenibilidad y claridad en la separación de responsabilidades. La comunicación se realiza mediante JSON, lo que facilita su integración con cualquier cliente moderno.

La API está diseñada para ser consumida por aplicaciones frontend, sistemas internos de la empresa, herramientas de automatización o clientes de prueba como Postman o cURL. Su diseño orientado a recursos permite que pueda integrarse fácilmente en flujos de trabajo corporativos o en aplicaciones más amplias que requieran gestionar disponibilidad de salas.

2. Arquitectura del proyecto

La API está organizada siguiendo una arquitectura por capas, lo que permite separar responsabilidades, facilitar el mantenimiento y garantizar que cada parte del sistema cumpla una función clara. Esta estructura también asegura que las reglas de negocio, la validación y el acceso a datos estén correctamente aislados.

- Arquitectura en capas:
    - **Controller**

      Ubicada en controlador/.

      Responsabilidad:

        - Exponer los endpoints REST.
        - Recibir las peticiones del cliente (JSON).
        - Validar datos básicos mediante anotaciones (@Valid).
        - Delegar toda la lógica al servicio.
        - Devolver DTOs, nunca entidades.

Ejemplos en tu proyecto:

- ReservaControlador.java
- SalaControlador.java
- UsuarioControlador.java

- **Service**

  Ubicada en servicio/.

  Responsabilidad:

  - Contiene la lógica de negocio real.
  - Aplica todas las reglas del dominio:
  - Validación de fechas.
  - Evitar solapamientos.
  - Control de estados (PENDING, CONFIRMED, CANCELLED).
  - Restricciones como “no editar reservas confirmadas”, “no cancelar reservas finalizadas”, etc.
  - Lanza excepciones personalizadas cuando algo no cumple las reglas.

  Ejemplo en tu proyecto:

  - ReservaServicio.java

- **Repository**

  Ubicación: repositorio/

  Responsabilidad:

  - Acceso a la base de datos mediante Spring Data JPA.
  - CRUD de entidades.
  - Consultas personalizadas (por ejemplo, detectar solapamientos).

  Repositorios del proyecto:

  - Repositorio_Reserva.java
  - Repositorio_Sala.java
  - Repositorio_Usuario.java

- **Domain (Entities)**

  Ubicación: modelo/

  Responsabilidad:

  - Representar las tablas de la base de datos.
  - Definir relaciones entre entidades.
  - Contener atributos persistentes.

  Entidades del proyecto:

  - Reserva.java
  - Sala.java
  - Usuario.java
  - StatusReserva.java (enum)

- **DTO**

  Ubicación: dto/

  Responsabilidad:

  - Controlar qué datos entran y salen de la API.
  - Evitar exponer entidades directamente.
  - Permitir validaciones específicas del JSON recibido.

  DTOs del proyecto:

  - ReservaRequestDTO.java
  - ReservaResponseDTO.java

  ¿Por qué no se exponen entidades directamente?

  - Porque contienen relaciones internas que no deben mostrarse.
  - Porque acoplaría el modelo de base de datos a la API.
  - Porque podría permitir modificar campos sensibles.
  - Porque los DTOs permiten validar y controlar la estructura del JSON recibido.

- **Mapper**

  Ubicación: mapper/

  Responsabilidad:

  - Convertir entidades ↔ DTOs.
  - Mantener el código limpio y evitar duplicación.

  Mapper del proyecto:

  - ReservaMapper.java

- **Exception handling**

  Ubicación: excepciones/

  Responsabilidad:

  - Gestionar errores de forma centralizada.
  - Devolver mensajes claros y códigos HTTP correctos.
  - Capturar excepciones personalizadas como:
  - FechasInvalidasException
  - ReservaSolapadaException
  - ReservaNoEncontradaException
  - UsuarioNoEncontradoException
  - SalaNoEncontradaException
  - ValidacionMultipleException

  Manejador global:

  - GlobalExceptionHandler.java
- **Config**

  Ubicación: config/

  Responsabilidad:

  - Configurar interceptores.
  - Validar el tipo de JSON recibido (usuario, sala o reserva).
  - Registrar interceptores en WebConfig.java.

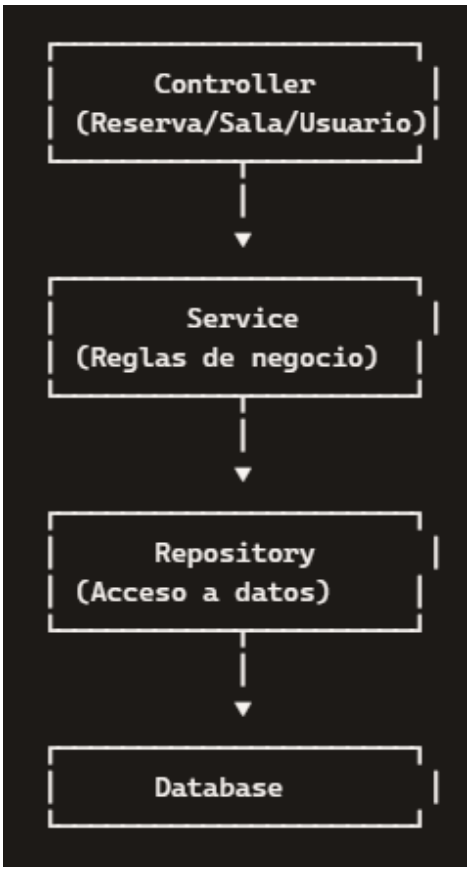
  Archivos clave:

  - TipoEsperadoInterceptor.java
  - WebConfig.java

```
Cliente → Controller → Service → Repository → Base de datos
                        ↓
                    Excepciones
                        ↓
                      DTOs
                        ↓
                    Respuesta
```

Explicación del flujo:

1. El Controller recibe el JSON y lo valida.
2. Luego convierte el JSON a un DTO.
3. Despues llama al Service, que aplica reglas de negocio.
4. El Service usa el Repository para consultar o modificar la base de datos.
5. El Repository devuelve entidades.
6. El Service las convierte a DTOs mediante el Mapper.
7. El Controller envía la respuesta al cliente.



3. Modelo de datos

La base de datos del proyecto está formada por tres tablas principales: salas, usuarios y reservas. Cada tabla corresponde directamente a una entidad JPA definida en el paquete modelo. El diseño es sencillo, claro y orientado a mantener la integridad de los datos mediante restricciones a nivel de base de datos y validaciones implementadas en la lógica de negocio.

Entidad Sala (Tabla: salas)

Campos

- id_sala — BIGINT, PK, autoincremental
- nombre_sala — VARCHAR, NOT NULL
- capacidad_sala — INT, NOT NULL

Restricciones

- nombre_sala es obligatorio (NOT NULL).
- capacidad_sala es obligatoria (NOT NULL).
- No existe restricción de unicidad en el nombre (según tu código actual).

Descripción

Representa una sala disponible para reservas.

No tiene relaciones JPA directas, pero se relaciona con la tabla reservas mediante el campo id_sala.

Entidad Usuario (Tabla: usuarios)

Campos

- id_usuario — BIGINT, PK, autoincremental
- nombre_usuario — VARCHAR, NOT NULL
- email_usuario — VARCHAR, NOT NULL

Restricciones

- Ambos campos son NOT NULL.
- No existe restricción de unicidad en el email (según tu código actual).

Descripción

Representa a un usuario que puede realizar reservas.

Se relaciona con la tabla reservas mediante el campo id_usuario.

Entidad Reserva (Tabla: reservas)

Campos

- id_reserva — BIGINT, PK, autoincremental
- id_sala — BIGINT, NOT NULL
- id_usuario — BIGINT, NOT NULL
- fecha_inicio_reserva — DATE, NOT NULL
- fecha_fin_reserva — DATE, NOT NULL
- estatus_reserva — ENUM (PENDIENTE, CONFIRMADA, CANCELADA), NOT NULL

Relaciones

Tu diseño no usa relaciones JPA (@ManyToOne), sino que almacena directamente los IDs:

- id_sala → referencia lógica a salas.id_sala
- id_usuario → referencia lógica a usuarios.id_usuario

Esto implica:

- No existen claves foráneas reales en la base de datos.
- La integridad se controla desde el servicio (ReservaServicio).

Restricciones

- Todas las columnas son NOT NULL.
- estatus_reserva se almacena como texto gracias a @Enumerated(EnumType.STRING).
- Las fechas deben cumplir reglas de negocio:
    - fecha_inicio_reserva < fecha_fin_reserva
    - No se permiten solapamientos
    - No se puede editar una reserva confirmada
    - No se puede cancelar una reserva finalizada
    - No se puede confirmar una reserva ya iniciada

Estas reglas están implementadas en ReservaServicio.

Reglas de negocio aplicadas al modelo

Aunque la base de datos no define claves foráneas ni restricciones complejas, tu servicio sí implementa validaciones estrictas:

- Validación de fechas
    - Inicio < Fin
    - No pueden ser iguales
    - No pueden ser nulas
- Solapamientos
  Implementado mediante la consulta JPQL en Repositorio_Reserva:

```
SELECT r FROM Reserva r

WHERE r.id_sala = :idSala

AND r.fecha_inicio_reserva <= :fechaFin

AND r.fecha_fin_reserva >= :fechaInicio
```

- Estados de reserva
    - No editar reservas confirmadas
    - No cancelar reservas finalizadas
    - No confirmar reservas ya iniciadas
    - No confirmar/cancelar dos veces

Estas reglas garantizan la integridad del sistema aunque la base de datos sea sencilla.

Índices

H2 y JPA generan automáticamente índices sobre:

- Claves primarias (id_sala, id_usuario, id_reserva)
- Campos usados en consultas (id_sala, id_usuario)

No has definido índices manuales, pero los automáticos son suficientes para el tamaño del proyecto.

Relaciones entre tablas

Aunque no están definidas con anotaciones JPA, las relaciones lógicas son:

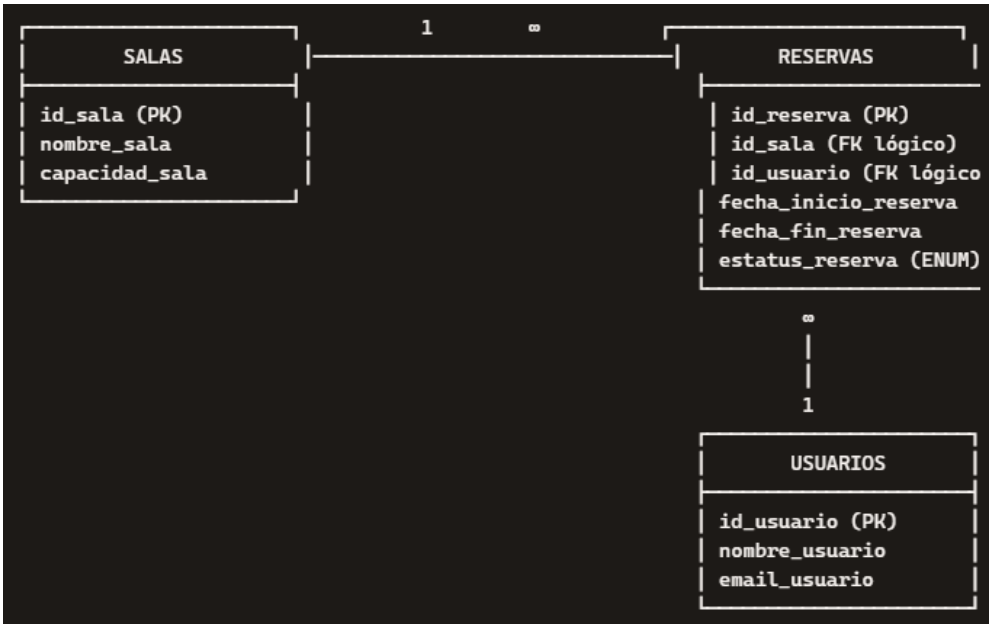
Sala 1 —— ∞ Reserva

Una sala puede tener muchas reservas.

Usuario 1 —— ∞ Reserva

Un usuario puede realizar muchas reservas.

Diagrama Entidad–Relación (ER)



4. Reglas de negocio

La API implementa un conjunto de reglas de negocio que garantizan la integridad del sistema de reservas. Estas reglas están centralizadas en la capa de servicio, principalmente en ReservaServicio, y se apoyan en consultas del repositorio Repositorio_Reserva. A continuación se describen las reglas más importantes y cómo se implementan en el código.

Validación de reservas solapadas

Regla

No puede existir una reserva cuyo rango de fechas se solape con otra reserva ya existente en la misma sala.

Cómo se implementa

La validación se realiza en la capa Service, dentro de ReservaServicio, utilizando una consulta personalizada del repositorio:

```
@Query("""
   SELECT r FROM Reserva r
   WHERE r.id_sala = :idSala
   AND r.fecha_inicio_reserva <= :fechaFin
   AND r.fecha_fin_reserva >= :fechaInicio
""")
List<Reserva> buscarSolapadas(...)
```

El servicio llama a este método antes de crear o editar una reserva:

- Si la lista devuelta no está vacía, significa que existe un solapamiento.
- En ese caso se lanza ReservaSolapadaException.

Resultado

La API devuelve un 409 CONFLICT con el mensaje:

"Ya existe una reserva en esa sala para ese rango de fechas"

Validación de fechas: endDateTime > startDateTime

Regla

- La fecha de fin debe ser posterior a la fecha de inicio.
- No se permiten reservas de 0 días ni fechas invertidas.

Cómo se implementa

En ReservaServicio se comprueba:

```
if (fechaInicio == null || fechaFin == null || !fechaInicio.isBefore(fechaFin)) {

    throw new FechasInvalidasException("La fecha de inicio debe ser anterior a la fecha de fin");

}
```

Resultado

La API devuelve un 400 BAD REQUEST con el mensaje:

"La fecha de inicio debe ser anterior a la fecha de fin"

Estados de una reserva

El estado se almacena en el campo estatus_reserva de la entidad Reserva:

```
public enum StatusReserva {

    PENDIENTE,

    CONFIRMADA,

    CANCELADA

}
```

Estados posibles

- PENDIENTE: Estado inicial de toda reserva.
- CONFIRMADA: La reserva queda bloqueada y no puede modificarse.
- CANCELADA: La reserva queda anulada.

Transiciones válidas entre estados

Estado actual	Acción	¿Permitido?	Resultado
PENDIENTE	Confirmar	Sí	Pasa a CONFIRMADA
PENDIENTE	Cancelar	Sí	Pasa a CANCELADA
CONFIRMADA	Cancelar	Sí	Pasa a CANCELADA
CONFIRMADA	Editar	No	Error
CANCELADA	Confirmar	No	Error
CANCELADA	Editar	Sí	Permitido (según tu lógica actual)

Intentar confirmar una reserva ya confirmada

Regla

Una reserva no puede confirmarse dos veces.

Cómo se implementa

En ReservaServicio:

java

```
if (reserva.getEstatus_reserva() == StatusReserva.CONFIRMADA) {
    throw new IllegalStateException("La reserva ya está confirmada");
}
```

Resultado

La API devuelve un 400 BAD REQUEST con el mensaje:

"La reserva ya está confirmada"

Intentar cancelar una reserva ya cancelada

Regla

Una reserva no puede cancelarse dos veces.

Cómo se implementa

En ReservaServicio:

```
if (reserva.getEstatus_reserva() == StatusReserva.CANCELADA) {
    throw new IllegalStateException("La reserva ya está cancelada");
}
```

Resultado

La API devuelve un 400 BAD REQUEST con el mensaje:

"La reserva ya está cancelada"

No se puede confirmar una reserva que ya ha comenzado

Regla

Si la fecha de inicio es anterior a hoy, la reserva ya está en curso y no puede confirmarse.

Cómo se implementa

En ReservaServicio:

```
if (reserva.getFecha_inicio_reserva().isBefore(LocalDate.now())) {

    throw new IllegalStateException("No se puede confirmar una reserva que ya ha comenzado");

}
```

No se puede cancelar una reserva que ya ha finalizado

Regla

Si la fecha de fin ya pasó, la reserva está terminada y no puede cancelarse.

Cómo se implementa

En ReservaServicio:

```
if (reserva.getFecha_fin_reserva().isBefore(LocalDate.now())) {

    throw new IllegalStateException("No se puede cancelar una reserva que ya ha finalizado");

}
```

No se puede editar una reserva confirmada

Regla

Una reserva confirmada queda bloqueada.

Cómo se implementa

En ReservaServicio:

```
if (reserva.getEstatus_reserva() == StatusReserva.CONFIRMADA) {

    throw new IllegalStateException("No se puede editar una reserva confirmada");

}
```

No se puede borrar una reserva confirmada

Regla

Las reservas confirmadas no pueden eliminarse.

Cómo se implementa

En ReservaServicio:

```
if (reserva.getEstatus_reserva() == StatusReserva.CONFIRMADA) {

    throw new IllegalStateException("No se puede borrar una reserva confirmada");

}
```

5. Documentación de endpoints

La API expone endpoints para gestionar salas, usuarios y reservas. Cada endpoint incluye su método HTTP, URL, descripción, ejemplo de petición, ejemplo de respuesta y códigos de estado posibles.

Endpoints de Salas

POST /salas

Descripción

Crea una nueva sala.

Request JSON (real)

```
{

  "nombre_sala": "Sala Azul",

  "capacidad_sala": 20

}
```

Response JSON (real)

```
{

  "id_sala": 1,

  "nombre_sala": "Sala Azul",

  "capacidad_sala": 20

}
```

Códigos de respuesta

- 201 Created – Sala creada correctamente
- 400 Bad Request – JSON inválido
- 409 Conflict – Si implementaras unicidad (no en tu caso actual)

GET /salas

Descripción

Devuelve la lista de todas las salas.

Response JSON (real)

```
[
  {
    "id_sala": 1,
    "nombre_sala": "Sala Azul",
    "capacidad_sala": 20
  },
  {
    "id_sala": 2,
    "nombre_sala": "Sala Roja",
    "capacidad_sala": 15
  }
]
```

Códigos de respuesta

- 200 OK

Endpoints de Usuarios

POST /usuarios

Descripción

Crea un nuevo usuario.

Request JSON (real)

```
{
  "nombre_usuario": "Carlos Pérez",
  "email_usuario": "carlos@example.com"
}
```

Response JSON (real)

```
{
  "id_usuario": 1,
  "nombre_usuario": "Carlos Pérez",
  "email_usuario": "carlos@example.com"
}
```

Códigos de respuesta

- 201 Created
- 400 Bad Request

GET /usuarios

Descripción

Devuelve la lista de usuarios registrados.

Response JSON (real)

```
[
  {
    "id_usuario": 1,
    "nombre_usuario": "Carlos Pérez",
    "email_usuario": "carlos@example.com"
  }
]
```

Códigos de respuesta

- 200 OK

Endpoints de Reservas

Tus reservas usan DTOs, así que los JSON son exactamente los de tus clases ReservaRequestDTO y ReservaResponseDTO.

POST /reservas

Descripción

Crea una nueva reserva validando fechas, solapamientos y existencia de sala/usuario.

Request JSON (real)

```
{
  "sala_id": 1,
  "usuario_id": 1,
  "reserva_fecha_inicio": "2025-03-10",
  "reserva_fecha_fin": "2025-03-12",
  "estatus_reserva": "PENDIENTE"
}
```

Response JSON (real)

```
{
  "idReserva": 5,
  "idSala": 1,
  "idUser": 1,
  "fechaInicio": "2025-03-10",
  "fechaFin": "2025-03-12",
  "estatusReserva": "PENDIENTE"
}
```

Códigos de respuesta

- 201 Created – Reserva creada
- 400 Bad Request – Fechas inválidas
- 404 Not Found – Sala o usuario inexistente
- 409 Conflict – Reserva solapada

GET /reservas/{id}

Descripción

Obtiene una reserva por su ID.

Response JSON (real)

```
{
  "idReserva": 5,
  "idSala": 1,
  "idUser": 1,
  "fechaInicio": "2025-03-10",
  "fechaFin": "2025-03-12",
  "estatusReserva": "PENDIENTE"
}
```

Códigos de respuesta

- 200 OK
- 404 Not Found

PUT /reservas/{id}/confirmar

Descripción

Confirma una reserva si:

- No está ya confirmada
- No ha comenzado
- No está cancelada

Response JSON (real)

```
{
  "idReserva": 5,
  "idSala": 1,
  "idUser": 1,
  "fechaInicio": "2025-03-10",
  "fechaFin": "2025-03-12",
  "estatusReserva": "CONFIRMADA"
}
```

Códigos de respuesta

- 200 OK
- 400 Bad Request – Ya confirmada o ya iniciada
- 404 Not Found

PUT /reservas/{id}/cancelar

Descripción

Cancela una reserva si:

- No está ya cancelada
- No ha finalizado
- Response JSON (real)

```
{
  "idReserva": 5,
  "idSala": 1,
  "idUser": 1,
  "fechaInicio": "2025-03-10",
  "fechaFin": "2025-03-12",
  "estatusReserva": "CANCELADA"
}
```

Códigos de respuesta

- 200 OK
- 400 Bad Request – Ya cancelada o ya finalizada
- 404 Not Found

GET /reservas

Descripción

Lista todas las reservas.

Response JSON (real)

```
[
  {
    "idReserva": 5,
    "idSala": 1,
    "idUser": 1,
    "fechaInicio": "2025-03-10",
    "fechaFin": "2025-03-12",
    "estatusReserva": "PENDIENTE"
  }
]
```

Códigos de respuesta

- 200 OK

6. Validaciones

La API implementa validaciones tanto a nivel de DTOs como a nivel de reglas de negocio. Las validaciones se realizan mediante anotaciones de Jakarta Validation y se gestionan globalmente mediante un Exception Handler centralizado.

Anotaciones de validación utilizadas

En tu proyecto utilizas las siguientes anotaciones:

- @NotNull — obliga a que el campo no sea nulo.
- @FutureOrPresent — obliga a que la fecha sea hoy o futura.
- (No usas @Email, @NotBlank, @Positive, etc., porque tus entidades no lo requieren.)

Estas anotaciones están presentes en:

ReservaRequestDTO

```
@NotNull(message = "El id de la sala es obligatorio")

private Long sala_id;

@NotNull(message = "El id del usuario es obligatorio")

private Long usuario_id;

@NotNull(message = "La fecha de inicio de la reserva es obligatoria")

@FutureOrPresent(message = "La fecha de inicio debe ser hoy o una fecha futura")

private LocalDate reserva_fecha_inicio;

@NotNull(message = "La fecha de fin de la reserva es obligatoria")

@FutureOrPresent(message = "La fecha de fin debe ser hoy o una fecha futura")

private LocalDate reserva_fecha_fin;

@NotNull(message = "El estatus de la reserva es obligatorio")

private StatusReserva estatus_reserva;
```

Estas validaciones se aplican antes de entrar al servicio, directamente en el controlador.

¿Qué ocurre cuando fallan las validaciones?

Cuando un campo no cumple una validación:

1. Spring detecta el error automáticamente.
2. Lanza una excepción MethodArgumentNotValidException.
3. Tu clase GlobalExceptionHandler captura la excepción.
4. Se construye una respuesta JSON con:
    - mensaje general
    - lista de errores específicos
    - código HTTP 400

Ejemplo del manejador:

```
@ExceptionHandler(MethodArgumentNotValidException.class)
public ResponseEntity<Object> manejarValidaciones(MethodArgumentNotValidException ex) {
    List<String> errores = ex.getBindingResult()
            .getFieldErrors()
            .stream()
            .map(error -> error.getField() + ": " + error.getDefaultMessage())
            .toList();

    return ResponseEntity.badRequest().body(errores);
}
```

Cómo se devuelven los errores al cliente

Los errores se devuelven en formato JSON, con un listado de mensajes claros.

Ejemplo real de respuesta:

```
[
  "sala_id: El id de la sala es obligatorio",
  "reserva_fecha_inicio: La fecha de inicio debe ser hoy o una fecha futura"
]
```

El código HTTP devuelto es:

- 400 Bad Request

Ejemplo real de error de validación

Request enviado por el cliente

```
{
  "sala_id": null,
  "usuario_id": 1,
  "reserva_fecha_inicio": "2020-01-01",
  "reserva_fecha_fin": "2020-01-02",
  "estatus_reserva": "PENDIENTE"
}
```

Errores detectados

- sala_id es nulo → viola @NotNull
- reserva_fecha_inicio es una fecha pasada → viola @FutureOrPresent
- reserva_fecha_fin también es pasada → viola @FutureOrPresent

Respuesta JSON real

```
[
  "sala_id: El id de la sala es obligatorio",
  "reserva_fecha_inicio: La fecha de inicio debe ser hoy o una fecha futura",
  "reserva_fecha_fin: La fecha de fin debe ser hoy o una fecha futura"
]
```

Código HTTP

- 400 Bad Request

7. Manejo global de errores

La API implementa un sistema centralizado de gestión de errores mediante la anotación @ControllerAdvice, que permite capturar excepciones en un único punto y devolver respuestas coherentes, limpias y uniformes al cliente.

Qué es @ControllerAdvice

@ControllerAdvice es una anotación de Spring que permite:

- Interceptar excepciones lanzadas desde cualquier controlador.
- Centralizar el manejo de errores en una sola clase.
- Evitar respuestas 500 genéricas y poco claras.
- Devolver mensajes personalizados y códigos HTTP adecuados.
- Mantener la API consistente y profesional.

En tu proyecto, esta función la cumple la clase:

- GlobalExceptionHandler

Excepciones personalizadas del proyecto

Tu API define varias excepciones específicas para representar errores de negocio:

Errores 404 – Recurso no encontrado

- ReservaNoEncontradaException
- SalaNoEncontradaException
- UsuarioNoEncontradoException

Errores 400 – Datos incorrectos

- FechasInvalidasException
- ValidacionMultipleException
- IllegalArgumentException (capturada como 400)

Errores de JSON mal formado (HttpMessageNotReadableException, InvalidFormatException, etc.)

Errores 409 – Conflicto

- ReservaSolapadaException
  (cuando una reserva se solapa con otra existente)

Qué devuelve cada tipo de error

Tu GlobalExceptionHandler devuelve respuestas JSON muy claras, siempre con un mensaje dentro de un objeto:

400 – Bad Request (datos incorrectos)

Se devuelve cuando:

- El JSON está mal formado.
- Las fechas no son válidas.
- El cliente envía datos del tipo incorrecto.
- Hay múltiples errores de validación.

Formato real:

```
{
  "error": "Los datos enviados no corresponden a los que tienes que meter. Inténtelo de nuevo metiendo datos de reserva."
}
```

O en caso de validación múltiple:

```
{
  "errores": [
    "sala_id: El id de la sala es obligatorio",
    "reserva_fecha_inicio: La fecha de inicio debe ser hoy o una fecha futura"
  ]
}
```

404 – Not Found (recurso inexistente)

Se devuelve cuando:

- No existe la sala solicitada.
- No existe el usuario solicitado.
- No existe la reserva solicitada.

Formato real:

```
{
  "error": "No existe la reserva con ID: 10"
}
```

409 – Conflict (conflicto de negocio)

Se devuelve cuando:

- Se intenta crear o editar una reserva que se solapa con otra.

Formato real:

```
{
  "error": "Ya existe una reserva en esa sala para ese rango de fechas"
}
```

Ejemplo real de respuesta de error estándar

Petición incorrecta

```
{
  "sala_id": null,
  "usuario_id": 1,
  "reserva_fecha_inicio": "2020-01-01",
  "reserva_fecha_fin": "2020-01-02",
  "estatus_reserva": "PENDIENTE"
}
```

Respuesta devuelta por tu API

```
{
  "errores": [
    "sala_id: El id de la sala es obligatorio",
    "reserva_fecha_inicio: La fecha de inicio debe ser hoy o una fecha futura",
    "reserva_fecha_fin: La fecha de fin debe ser hoy o una fecha futura"
  ]
}
```

Código HTTP

- 400 Bad Request

8. Cómo ejecutar el proyecto

Este apartado explica cómo cualquier persona puede ejecutar la API de reservas de salas usando únicamente Java, Maven, H2 y Postman, tal como tú lo has trabajado.

Requisitos del sistema

Para ejecutar el proyecto se necesita:

- Java 17 (compatible con Spring Boot 3)
- Maven 3.8+
- Navegador web (para acceder a H2 Console)
- Postman o cualquier cliente REST

No se necesita instalar MySQL ni ninguna base de datos externa.

Base de datos utilizada: H2 (en memoria)

El proyecto usa H2, una base de datos ligera que se ejecuta en memoria mientras la aplicación está activa.

Ventajas:

- No requiere instalación.
- Se crea automáticamente al arrancar.
- Se borra al apagar la aplicación.
- Las tablas se generan solas a partir de las entidades.

Acceso a la consola H2

Una vez arrancado el proyecto, la consola está disponible en:

```
http://localhost:8080/h2-console
```

Configuración típica:

- JDBC URL: jdbc:h2:mem:testdb
- User: sa
- Password: (vacío)

Arrancar el proyecto

Opción 1: Ejecutar con Maven (recomendado)

```
mvn spring-boot:run
```

Opción 2: Ejecutar el JAR

Primero compilar:

```
mvn clean package
```

Luego ejecutar:

```
java -jar target/reservasalasapi-0.0.1-SNAPSHOT.jar
```

Puerto de ejecución

La API se ejecuta en el puerto 8080.

URL base:

```
http://localhost:8080
```

Configuración de la base de datos (H2)

En application.properties debe aparecer algo como:

```
spring.datasource.url=jdbc:h2:mem:testdb

spring.datasource.driverClassName=org.h2.Driver

spring.datasource.username=sa

spring.datasource.password=


spring.h2.console.enabled=true

spring.h2.console.path=/h2-console


spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
```

¿Qué significa esto?

- La base de datos se crea sola en memoria.
- Las tablas se generan automáticamente.
- No hace falta crear nada manualmente.
- Puedes ver los datos desde /h2-console.

Datos de prueba iniciales

El proyecto no incluye datos precargados, pero puedes crear registros fácilmente desde Postman.

Crear una sala

```
POST http://localhost:8080/salas

{

  "nombre_sala": "Sala Azul",

  "capacidad_sala": 20

}
```

Crear un usuario

```
POST http://localhost:8080/usuarios

{

  "nombre_usuario": "Carlos Pérez",

  "email_usuario": "carlos@example.com"

}
```

Crear una reserva

```
POST http://localhost:8080/reservas

{

  "sala_id": 1,

  "usuario_id": 1,

  "reserva_fecha_inicio": "2025-03-10",

  "reserva_fecha_fin": "2025-03-12",

  "estatus_reserva": "PENDIENTE"

}
```

Resumen rápido para arrancar

1. Instalar Java 17 y Maven.
2. Abrir terminal en la carpeta del proyecto.
3. Ejecutar:

   ```
   mvn spring-boot:run
   ```

4. Abrir la consola H2:

   ```
   http://localhost:8080/h2-console
   ```

5. Probar la API con Postman:

   ```
   GET http://localhost:8080/salas
   ```

9. Decisiones técnicas y mejoras

La API se ha construido tomando decisiones orientadas a la claridad, la separación de responsabilidades y la robustez del dominio. Este apartado explica las decisiones clave, su motivación y las mejoras que se implementarían con más tiempo.

Uso de DTOs: una decisión para proteger el dominio

Elegí utilizar DTOs (Data Transfer Objects) para separar claramente:

- Los datos que recibe la API.
- Los datos que maneja la lógica interna.
- Los datos que se devuelven al cliente.

Esta decisión evita exponer directamente las entidades JPA, lo cual aporta varias ventajas:

- Seguridad: el cliente no puede modificar campos internos que no deberían tocarse.
- Evolución del modelo: puedo cambiar la entidad sin romper la API pública.
- Validación más limpia: las anotaciones de validación viven en el DTO, no en la entidad.
- Control del formato de entrada/salida: puedo devolver solo lo necesario, no todo el objeto.

En resumen, los DTOs permiten que la API sea más estable, más segura y más fácil de mantener.

Validación de solapes: decisión basada en reglas de negocio reales

La validación de solapes se implementó en la capa de servicio porque:

- Es una regla de negocio, no una validación sintáctica.
- Depende del estado de la base de datos (reservas existentes).
- Requiere lógica temporal (comparación de rangos de fechas).
- No puede resolverse con anotaciones como @FutureOrPresent.

La decisión fue:

1. Consultar si existen reservas activas en la misma sala.
2. Comparar los rangos de fechas.
3. Lanzar ReservaSolapadaException si hay conflicto.

Esto permite:

- Mantener la integridad del sistema.
- Evitar reservas duplicadas.
- Centralizar la lógica en un único punto.

Es una solución simple, clara y eficaz para el alcance del proyecto.

Mejoras que implementaría con más tiempo

Una mejora muy importante ha sido la implementación de un sistema de autenticación y registro de usuarios, incluyendo:

Pantalla de registro

- Nombre
- Apellidos
- Usuario (user)
- Contraseña (pass)

Pantalla de login

- Usuario
- Contraseña
- Validación de campos vacíos y credenciales
- Acceso a la aplicación solo si son correctas

Beneficios de esta mejora

- Cada usuario gestionaría únicamente sus reservas.
- Se evitaría que cualquiera pueda crear reservas en nombre de otros.
- Se podría añadir roles (admin/usuario).
- Se podría extender a un sistema de permisos.
- Se podría añadir trazabilidad (quién creó qué).
- Se rescata el nombre de quien accede a la aplicación y realizar las reservas

Esta mejora convertiría la API en un sistema real de gestión de reservas, no solo en un backend funcional.

Limitaciones actuales de la API

Aunque la API es funcional y coherente, tiene algunas limitaciones naturales por el alcance del proyecto:

- **No hay autenticación ni autorización.**
  Cualquiera puede acceder a todos los endpoints.
- **No hay persistencia real si se usa H2 en memoria.**
  Los datos desaparecen al reiniciar la aplicación.
- **No hay paginación ni filtros avanzados.**
  Las listas pueden crecer demasiado en un entorno real.
- **No hay auditoría ni trazabilidad.**
  No se registra quién crea, modifica o elimina reservas.
- **La validación de solapes funciona bien, pero no contempla casos más complejos,** como reservas recurrentes o franjas horarias parciales.
- **No hay documentación automática (Swagger/OpenAPI),** lo cual sería útil para clientes externos.

Estas limitaciones no afectan al funcionamiento básico, pero sí a la escalabilidad y profesionalización del sistema.